05. Sorting

Hyunjong Choi, Ph.D.
Assistant Professor of Computer Science
San Diego State University

Overview

- ❑ Quicksort (Textbook 3rd edition 170 – 190)
- ❑ Heapsort (Textbook 3rd edition 151 – 169)
- ❑ Linear-time sorting (Textbook 3rd edition 191 – 212)

Quicksort: Divide-and-Conquer algorithm

Divide step

Partition (rearrange) the array $A[p:r]$ into two subarrays $A[p:q-1]$ and $A[q+1:r]$ such that **each element in $A[p:q-1] \leq A[q]$**, and **$A[q] \leq$ each element in $A[q+1:r]$**

Conquer step

Sort the two subarrays $A[p:q-1]$ and $A[q+1:r]$ by recursive calls

Combine step

No work is needed to combine the subarrays, because they are already sorted: the entire array $A[p:r]$ is now sorted.

QUICKSORT procedure

❑ Pseudocode

```
QUICKSORT(A, p, r)
1.    if p < r
2.          q = PARTITION(A, p, r)
3.          QUICKSORT(A, p, q − 1)
4.          QUICKSORT(A, q + 1, r)
```

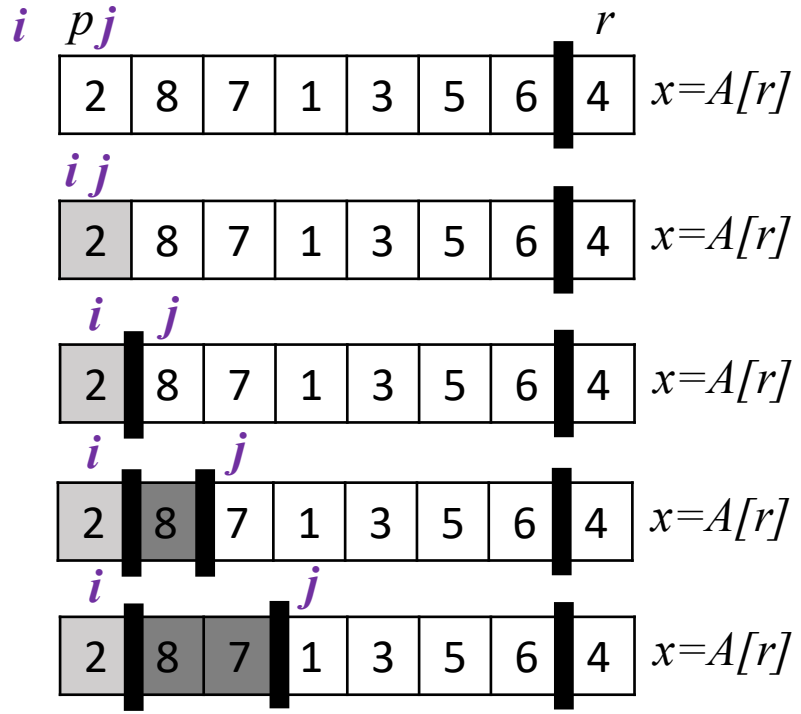
- To sort an entire array *A*, just call QUICKSORT(*A*, 1, *A.length*)
- The key is the PARTITION procedure

PARTITION procedure

❑ Partition (divide)

- Partition the array $A[p:r]$ into two subarrays $A[p:q-1]$ and $A[q+1:r]$ such that **each element in $A[p:q-1] \leq A[q]$**, and **$A[q] \leq$ each element in $A[q+1:r]$**.

```
PARTITION(A, p, r)
1.  x = A[r]
2.  i = p − 1
3.  for j = p to r − 1
4.        if A[j] ≤ x
5.              i = i + 1
6.              exchange A[i] with A[j]
7.  exchange A[i+1] with A[r]
8.  return i + 1
```

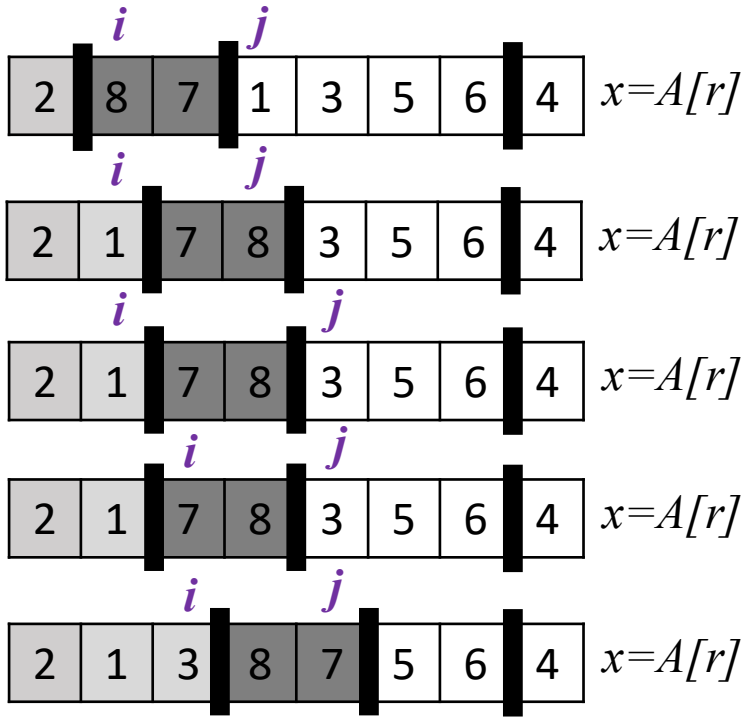


PARTITION procedure (cont.)

❑ Partition (divide)

- Partition the array $A[p:r]$ into two subarrays $A[p:q-1]$ and $A[q+1:r]$ such that **each element in $A[p:q-1] \le A[q]$**, and **$A[q] \le$ each element in $A[q+1:r]$**.

```
PARTITION(A, p, r)
1.   x = A[r]
2.   i = p − 1
3.   for j = p to r − 1
4.        if A[j] ≤ x
5.             i = i + 1
6.             exchange A[i] with A[j]
7.   exchange A[i+1] with A[r]
8.   return i + 1
```

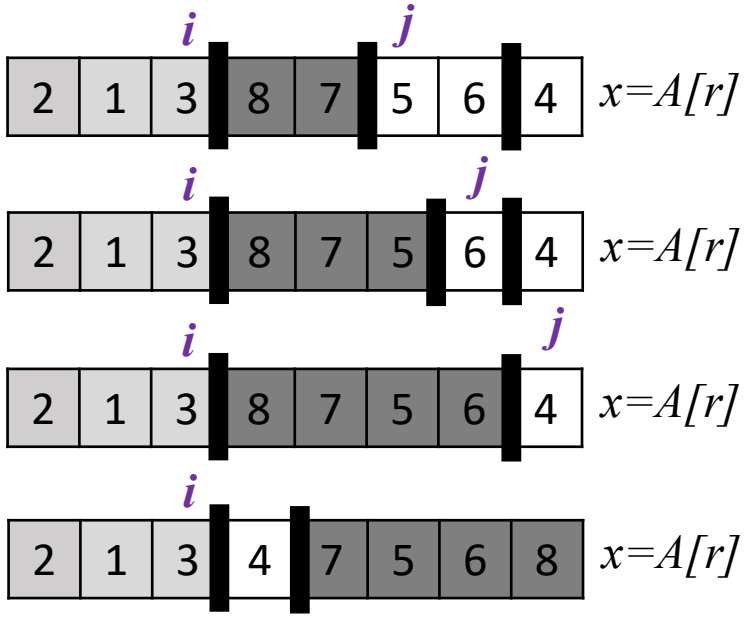


PARTITION procedure (cont.)

❑ Partition (divide)

- Partition the array $A[p:r]$ into two subarrays $A[p:q-1]$ and $A[q+1:r]$ such that **each element in $A[p:q-1] \leq A[q]$**, and **$A[q] \leq$ each element in $A[q+1:r]$**.

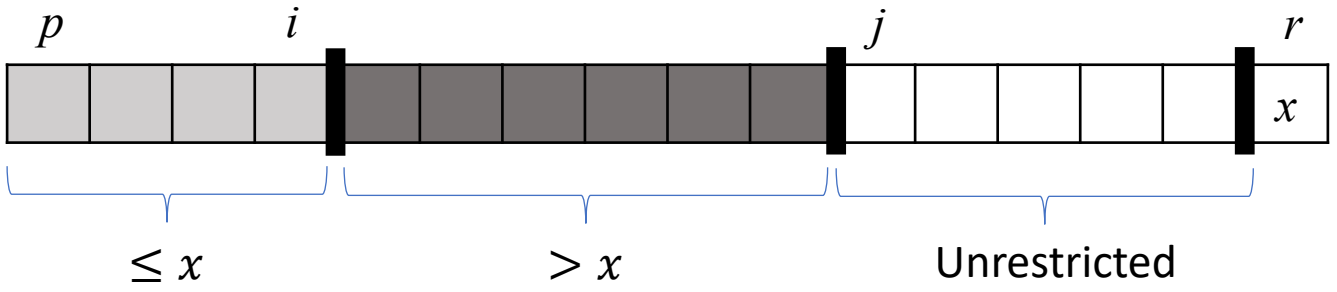
```
PARTITION(A, p, r)
1.   x = A[r]
2.   i = p − 1
3.   for j = p to r − 1
4.        if A[j] ≤ x
5.             i = i + 1
6.             exchange A[i] with A[j]
7.   exchange A[i+1] with A[r]
8.   return i + 1
```



Then the pivot's new index $(i+1)$ is returned

Summary of PARTITION procedure

- PARTITION procedure selects the last element $x = A[r]$ as a ***pivot*** element.
- As the procedure runs, it partitions the array into four (possibly empty) regions.



- At the start of the iteration of the for loop, for any index k:
  - If $p \leq k \leq i$, then $A[k] \leq x$.
  - If $i + 1 \leq k \leq j - 1$, then $A[k] > x$.
  - If $k = r$, then $A[k] = x$.

➡ These properties are the **loop invariant**.

Analyze loop invariant (optional)

Optional

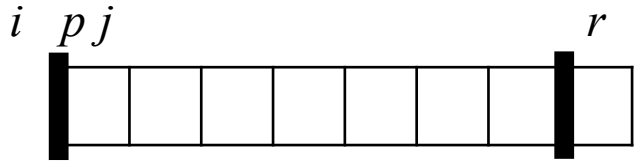
```
PARTITION(A, p, r)
1.    x = A[r]
2.    i = p − 1
3.    for j = p to r − 1
4.          if A[j] ≤ x
5.                i = i + 1
6.                exchange A[i] with A[j]
7.    exchange A[i+1] with A[r]
8.    return i + 1
```

- Loop invariant

1. If $p \le k \le i$, then $A[k] \le x$.
2. If $i + 1 \le k \le j - 1$, then $A[k] > x$.
3. If $k = r$, then $A[k] = x$.

❑ Initialization

- Prior to the first iteration, $i = p - 1$ and $j = p$, condition 1 and 2 are trivially satisfied, because no elements lie between p and i, nor between $i + 1$ and $j - 1$.



Analyze loop invariant (optional)

Optional

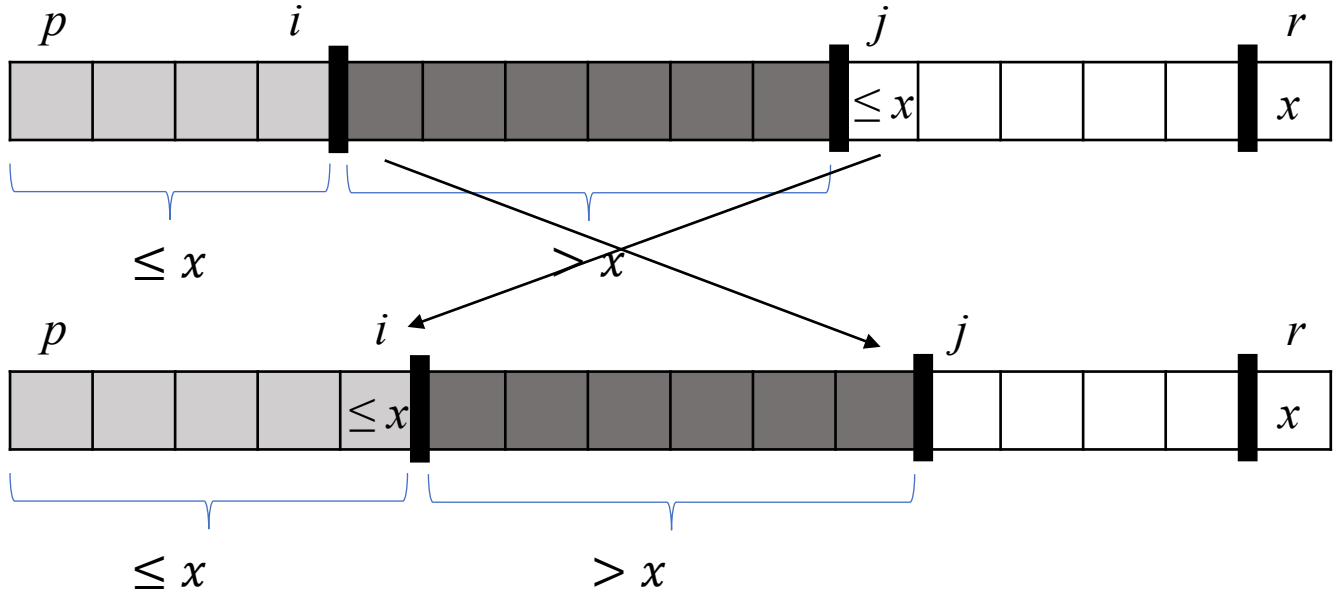
```
PARTITION(A, p, r)
1.    x = A[r]
2.    i = p − 1
3.    for j = p to r − 1
4.        if A[j] ≤ x
5.            i = i + 1
6.            exchange A[i] with A[j]
7.    exchange A[i+1] with A[r]
8.    return i + 1
```

- Loop invariant

1. If $p \le k \le i$, then $A[k] \le x$.
2. If $i + 1 \le k \le j - 1$, then $A[k] > x$.
3. If $k = r$, then $A[k] = x$.

❑ Maintenance

- Depending on the outcome of the **if** test in line 4, we consider two cases.
- If $A[j] \le x$, the loop increments i, swaps $A[i]$ and $A[j]$, and then increments j. We now have $A[i] \le x$, which satisfies condition 1, and $A[j-1] > x$, which satisfies condition 2.



Analyze loop invariant (optional)

Optional

```
PARTITION(A, p, r)
1.    x = A[r]
2.    i = p − 1
3.    for j = p to r − 1
4.          if A[j] ≤ x
5.                i = i + 1
6.                exchange A[i] with A[j]
7.    exchange A[i+1] with A[r]
8.    return i + 1
```

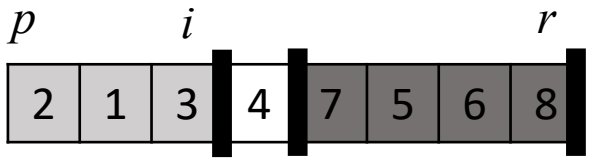
- Loop invariant

1. If $p \le k \le i$, then $A[k] \le x$.
2. If $i + 1 \le k \le j - 1$, then $A[k] > x$.
3. If $k = r$, then $A[k] = x$.

❑ Termination

- At termination, $j = r$. Every entry in the array is in one of the three conditions of the loop invariant. We have partitioned the values in the array into three sets: those less than or equal to x, those greater than x, and a singleton set containing x.

- The final two lines are critical: they move the pivot into its correct place, and return the pivot's new index.



Running time of PARTITION procedure

- The running time of PARTITION on $A[p:r]$ is $\Theta(n)$, where $n = r - p + 1$.
  - Because it uses one for loop that iterates from p to $r - 1$.

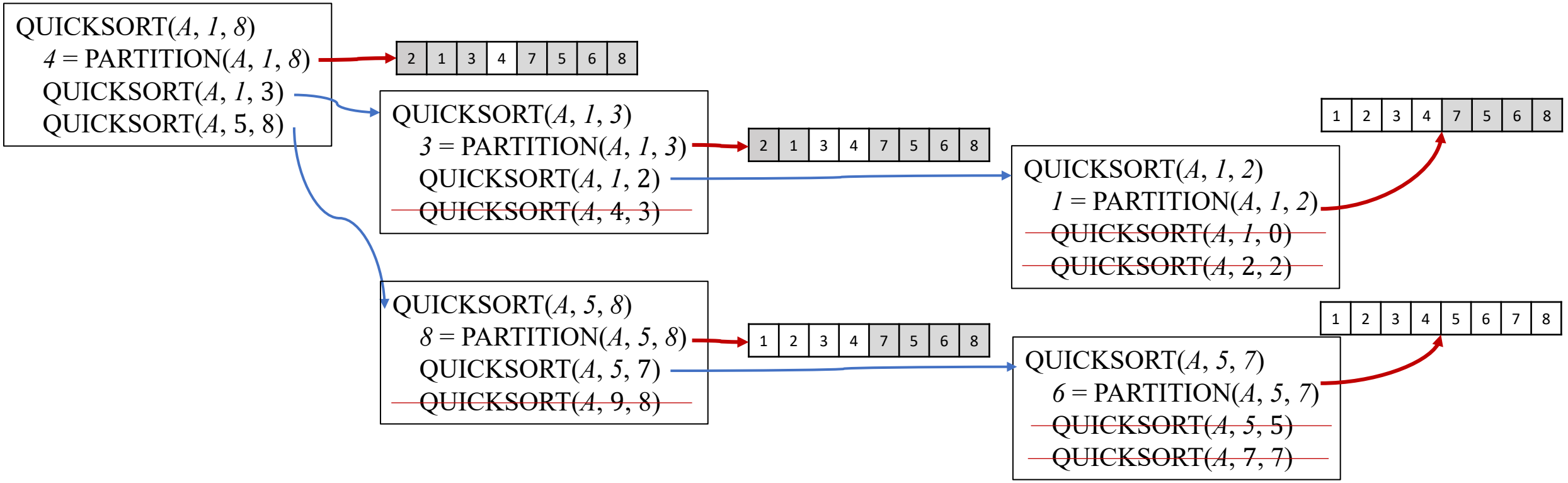
```
PARTITION(A, p, r)
1.    x = A[r]
2.    i = p − 1
3.    for j = p to r − 1
4.          if A[j] ≤ x
5.                i = i + 1
6.                exchange A[i] with A[j]
7.    exchange A[i+1] with A[r]
8.    return i + 1
```

QUICKSORT

```
QUICKSORT(A, p, r)
1.    if p < r
2.        q = PARTITION(A, p, r)
3.        QUICKSORT(A, p, q − 1)
4.        QUICKSORT(A, q + 1, r)
```

- QUICKSORT pseudocode
  - The result from the initial partition step is “4”

2	1	3	4	7	5	6	8



Running time of QUICKSORT procedure

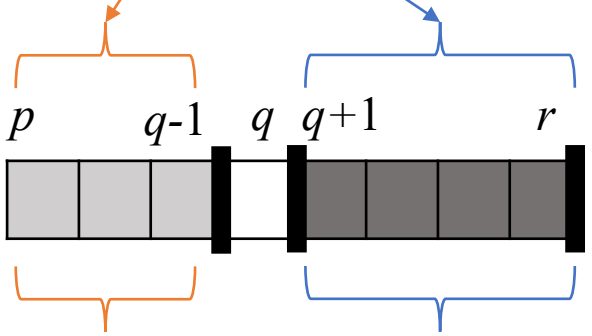
- The running time of QUICKSORT depends on the results of partitioning.
  - Recurrence equations could be
    - ✓ $T(n) = T\left(\frac{n}{2}\right) + T\left(\frac{n}{2}\right) + cn$
    - ✓ $T(n) = T\left(\frac{3n}{4}\right) + T\left(\frac{n}{4}\right) + cn$
    - .....
    - ✓ $T(n) = T\left(\frac{9n}{10}\right) + T\left(\frac{n}{10}\right) + cn$
    - ✓ $T(n) = T(n-1) + T(1) + cn$

```
QUICKSORT(A, p, r)
1.   if p < r
2.        q = PARTITION(A, p, r)
3.        QUICKSORT(A, p, q − 1)
4.        QUICKSORT(A, q + 1, r)
```

- **Balanced**: quicksort runs as fast as merge sort.
- **Unbalanced**: it can run as slowly as insertion sort.

Worst-case partitioning (Unbalanced)

```
QUICKSORT(A, p, r)
1.    if p < r
2.        q = PARTITION(A, p, r)
3.        QUICKSORT(A, p, q − 1)
4.        QUICKSORT(A, q + 1, r)
```



Length: $n_1 = q - p$

Length: $n_2 = r - q$

$n_1 + n_2 = r - p = n - 1$

- The worst-case partitioning: one subarray has $n - 1$ elements while the other has 0 elements.
  - ✓ Assume this extreme case occurs in each recursive call
  - ✓ PARTITION costs $\Theta(n) = cn$ time, $T(0) = \Theta(1)$,

$T(n) = T(n-1) + T(0) + cn$

$= T(n-1) + cn$, in which c is some constant

Worst-case partitioning (cont.)

- $T(n) = T(n-1) + cn$
  - Can we use master theorem?
  - A guess: $T(n) = T(n-1) + cn = T(n-2) + c(n-1) + cn$
    $= T(1) + 2c + 3c + \cdots + (n-1)c + nc$
  - The sum of $1c + 2c + 3c + \cdots + (n-1)\mathrm{c} + nc$ is about $\Theta(n^2)$.
- We can get $T(n) = \mathrm{O}(n^2)$.
  - Using substitution method, assume $T(n-1) \leq c(n-1)^2$, then

$$T(n) = T(n-1) + cn \leq c(n-1)^2 + dn$$
$$\leq cn^2$$
$$\Rightarrow \quad c(n^2 - 2n + 1) + dn \leq cn^2$$
$$\Rightarrow \quad c \geq \frac{n}{2n-1} d$$

- We want this inequality to always hold
- Boundary condition when n = 1

The guess $T(n) = O(n^2)$ is correct.

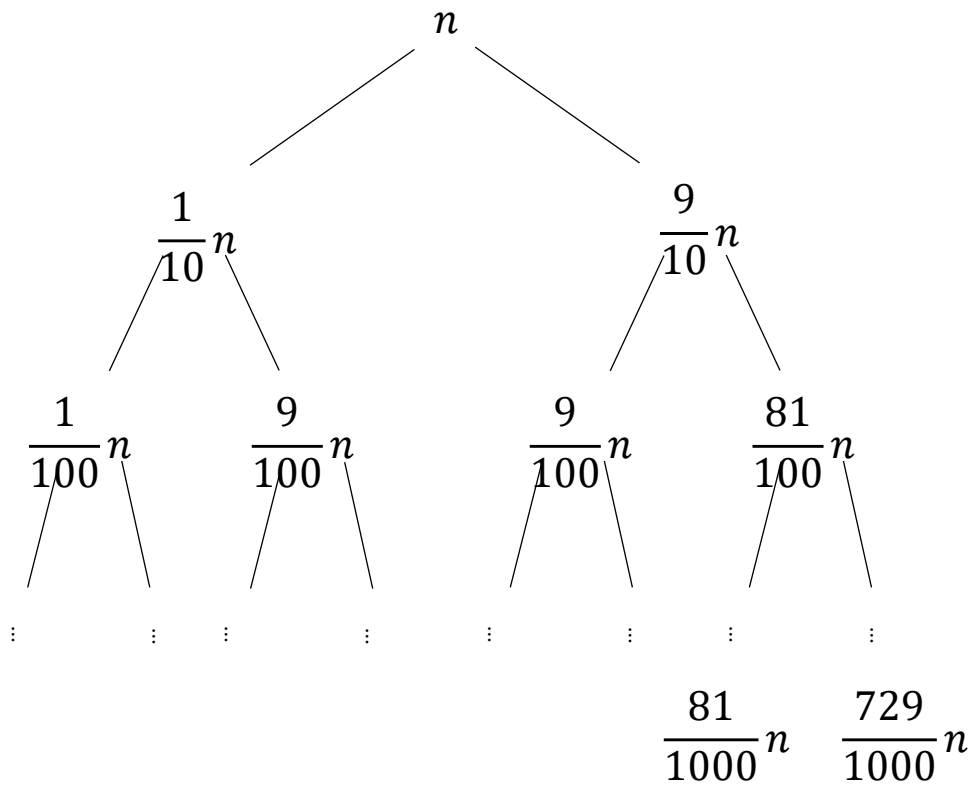
  - Thus, the inequality holds as long as $c \geq d$, and for $n > n_0 = 1$

Best-case partitioning

- The most even split produces two subarrays of lengths $\lfloor n/2 \rfloor$ and $\lfloor n/2 \rfloor + 1$.
  - i.e., $T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n)$

- By case 2 of master theorem, the solution is $\boldsymbol{T(n) = \Theta(n \lg n)}$.

What about average-case ?



- The average-case is much closer to the best-case than the worst-case
- Consider the partitioning that always produces a 9-to-1 proportional split:

$$T(n) = T\left(\frac{9n}{10}\right) + T\left(\frac{n}{10}\right) + cn$$

- The left-most branch reaches boundary when $\frac{1}{10^i}n = 1$, i.e., at depth $i = \log_{10} n$.
- The right-most branch reaches boundary when $\left(\frac{9}{10}\right)^i n = 1$, i.e., at depth $i = \log_{10/9} n$.
- The cost for each row is the same: n
- Therefore, $T(n) = O(n \lg n)$ (can prove by substitution method)

Constant proportional split is good

- Thus, with a 9-to-1 split at every level, quicksort still runs in $O(n \lg n)$ time which is asymptotically the same as if the best-case split.
- Any split of ***constant*** **proportionality** yields a recursion tree of depth of $\Theta(\lg n)$, and the cost of each level is $O(n)$.
- $T(n) = O(n \lg n)$ whenever the split has constant proportionality.

Randomized version of quicksort

- In order to make the split as balanced as possible,
  - **Randomly** choose the pivot instead of always choosing $A[r]$ during the partitioning stage.

```
RANDOMIZED-PARTITION(A, p, r)
1.    i = RANDOM(p, r)
2.    exchange A[r] with A[i]
3.    return PARTITION(A, p, r)
```

```
RANDOMIZED-QUICKSORT(A, p, r)
1.    if p < r
2.          q = RANDOMIZED-PARTITION(A, p, r)
3.          QUICKSORT(A, p, q − 1)
4.          QUICKSORT(A, q + 1, r)
```

Pros and cons of quicksort

Optional

❑ **Advantages:**

- In place sorting: does not use additional storage space (unlike merge sort)
- Efficient average case.

❑ **Disadvantage:**

- Needs to choose a proper pivot element in the partition step.
- Large *stack* usage.

Example

Problem

- Q1. How would you modify QUICKSORT to sort into nonincreasing order?

- Q2. What value of q does PARTITION return when all elements in the array $A[p..r]$ have the same value?